# Integrated System Architectures

LAB3: Frequently Asked Questions

Luigi Giuffrida and Alessandra Dolmeta

**PLEASE:** Before starting the lab, carefully read the entire documentation available on "Portale della Didattica". Many common issues are already addressed there. The tools also provide useful information on the command line, including details about errors, warnings, ... please take a moment to check these messages. If, after reading the material and the tool outputs, something is still unclear, do not hesitate to contact us for further questions.

## Problem dimensions and need for tiling

In this lab, the accelerator must implement a multi–channel 1D convolution with:

- at least $C_{\text{in}} = 16$ input channels;
- $C_{\text{out}} = 8$ output channels;
- kernel length $K = 5$ (five coefficients per input–output channel pair).

Each kernel coefficient must be treated as independent: there are no symmetries that would allow one to reuse or mirror coefficients. Therefore, for each pair of input channel and output channel there are $K = 5$ distinct coefficients.

Assume each input channel has length $L_{\text{in}}$ (for instance $L_{\text{in}} = 128$ samples as in the template). The length of each output channel is:

$$L_{\text{out}} = \begin{cases} L_{\text{in}} & \text{with zero padding} \\ L_{\text{in}} - K + 1 = L_{\text{in}} - 4 & \text{without padding ("valid" convolution)} \end{cases}$$

Data are stored in the scratchpad as 32-bit words. With a scratchpad size of 512 B, this corresponds to:

$$N_{\text{SP}} = \frac{512 \text{ bytes}}{4 \text{ bytes/word}} = \mathbf{128} \text{ 32-bit words.}$$

### Operations per output sample

Consider one specific output channel $j \in \{0, \ldots, C_{\text{out}} - 1\}$ and one specific output sample index $n \in \{0, \ldots, L_{\text{out}} - 1\}$. The multi–channel 1D convolution can be written as:

$$y_j[n] = \sum_{c=0}^{C_{\text{in}}-1} \sum_{k=0}^{K-1} x_c[n+k] \cdot h_{j,c}[k]$$

where

- $x_c[\cdot]$ is the input signal of channel $c$,
- $h_{j,c}[k]$ is the $k$-th kernel coefficient connecting input channel $c$ to output channel $j$,
- $y_j[n]$ is the $n$-th output sample of output channel $j$.

The number of products needed for a *single* output sample $y_j[n]$ is:

$$N_{\text{mul, per sample}} = C_{\text{in}} \cdot K.$$

In this lab,

$$C_{\text{in}} = 16, \qquad K = 5 \quad \Rightarrow \quad N_{\text{mul, per sample}} = 16 \cdot 5 = 80.$$

These 80 products must then be accumulated into a single sum.
Summing $N$ terms requires $N - 1$ additions in a straightforward adder tree or serial accumulation:

$$N_{\text{add, per sample}} = N_{\text{mul, per sample}} - 1 = 80 - 1 = 79.$$

Therefore, for **each output sample** $y_j[n]$:

| 80 multiplications and 79 additions are required. |
|---|

The lab also constrains the accelerator datapath to use at most:

- 4 **hardware multipliers**;
- 5 **hardware adders**.

This means the 80 products and 79 sums for each output sample cannot be computed in full parallel fashion. Instead, the accelerator must reuse the limited multipliers and adders over multiple cycles, carefully scheduling the operations.

**Total number of data (inputs, outputs, kernels)**

If we take the minimum configuration required by the assignment:

$$C_{\text{in}} = 16, \quad C_{\text{out}} = 8, \quad K = 5, \quad L_{\text{in}} = 128,$$

and consider the padded case $L_{\text{out}} = 128$, we obtain:

$$\begin{aligned} \#\text{ input samples} &= C_{\text{in}} \cdot L_{\text{in}} = 16 \cdot 128 = 2048, \\ \#\text{ kernel coefficients} &= C_{\text{out}} \cdot C_{\text{in}} \cdot K = 8 \cdot 16 \cdot 5 = 640, \\ \#\text{ output samples} &= C_{\text{out}} \cdot L_{\text{out}} = 8 \cdot 128 = 1024. \end{aligned}$$

In total, the convolution involves

$$2048 + 640 + 1024 = 3712$$

32-bit words of data, while the scratchpad can hold only

$$N_{\text{SP}} = 128 \text{ words}.$$

This is over ***29 times*** more data than what fits into the scratchpad, so it is clearly impossible to keep all inputs, all kernels, and all outputs in the scratchpad at the same time.

## Why tiling is necessary

Because the scratchpad capacity is limited to 512 B (128 32-bit words), the accelerator must operate on *tiles* of the data rather than on the whole problem at once. Tiling means that we select smaller subproblems that fit in the scratchpad, compute partial results, and then move data back and forth to main memory as needed.

Moreover, it is not realistic (and not in the spirit of the assignment) to instantiate a very large number of registers after the scratchpad to simulate another large on-chip memory. For instance, having another "scratchpad" of 512 B worth of registers would defeat the purpose of the memory constraint. A reasonable design should use only a modest number of registers (for pipeline stages, partial sums, loop counters, etc.) and rely on the scratchpad as the main local storage.

#### Example tiling strategy #1: small time window, few channels

One possible tiling strategy is to process:

- a small number of input channels $C_{\text{in}}^{(t)}$ at a time;
- a small time window of $T$ output samples at a time;
- a small number of output channels $C_{\text{out}}^{(t)}$ at a time.

As an example, consider:

$$C_{\text{in}}^{(t)} = 4, \quad C_{\text{out}}^{(t)} = 2, \quad T = 8.$$

We need to store in the scratchpad, for this tile:

- input samples for $C_{\text{in}}^{(t)}$ channels over a window of $T$ samples plus the "halo" of $K - 1 = 4$ extra samples:

$$N_{\text{in}}^{(t)} = C_{\text{in}}^{(t)} \cdot (T + K - 1) = 4 \cdot (8 + 4) = 4 \cdot 12 = 48 \text{ words};$$

- kernel coefficients for $C_{\text{out}}^{(t)}$ output channels and $C_{\text{in}}^{(t)}$ input channels:

$$N_{\text{ker}}^{(t)} = C_{\text{out}}^{(t)} \cdot C_{\text{in}}^{(t)} \cdot K = 2 \cdot 4 \cdot 5 = 40 \text{ words};$$

- output samples of the tile:

$$N_{\text{out}}^{(t)} = C_{\text{out}}^{(t)} \cdot T = 2 \cdot 8 = 16 \text{ words}.$$

The total scratchpad usage for this tile is:

$$N_{\text{tile}} = N_{\text{in}}^{(t)} + N_{\text{ker}}^{(t)} + N_{\text{out}}^{(t)} = 48 + 40 + 16 = 104 \text{ words},$$

which is below the limit of 128 words. The accelerator can thus:

1. load this tile from main memory into the scratchpad;
2. compute the partial outputs for these $T$ time steps, 4 input channels, and 2 output channels;
3. write the results back to main memory (or accumulate them in place);
4. move to the next tile.

**Q1) Are the input channel data provided directly by the Croc SoC? Is the `sbr_obi*` signal a bus coming from the main architecture that directly targets the scratchpad memory?**

**A1)** The signal in question is already exposed in the accelerator wrapper:
`input croc_pkg::sbr_obi_req_t obi_req_i,`
`output croc_pkg::sbr_obi_rsp_t obi_rsp_o.`
This interface is connected to the scratchpad (SP) memory through a multiplexer, which allows selecting whether read and write requests to the SP memory are arbitrated by the microcontroller or by the accelerator.

**Q2) Can the output channel data be stored in registers instead of the scratchpad memory, using the scratchpad only for tiling?**

**A2)** No. The outputs must be written into the scratchpad (SP) memory, and therefore they must be taken into account in the tiling strategy. Instantiating all the registers required to hold the complete output does not make sense; the SP memory is the intended storage for the output data.

**Q3) Does the address counter (and its internal adder) used to generate scratchpad addresses count towards the limited number of adders allowed in the accelerator?**

**A3)** No. The counter (and its internal adder) used to compute addresses does not count towards the limited number of adders specified for the accelerator datapath. You may implement such a counter without it being included in the adder budget.

**Q4) Is the scratchpad memory synchronous or asynchronous, and can it be read and written at the same time?**

**A4)** The scratchpad (SP) memory is a synchronous memory with:

- one read port;
- one write port;

However, the memory interface exposed inside the accelerator provides only a single address port, so it is not possible to perform a read and a write in the same cycle through that interface. It is possible to write into the SP while the accelerator is performing computations, but **you must implement the required logic to manage potential conflicts**. In practice, the probability that a read and a write target the same location in the same cycle is low, but not zero.
A safe way to overlap computation and data transfers is to use **double buffering**: partition the SP into two regions and perform a ping-pong between them, using one region for computation while the other is being updated.

**Q5) How are the main memory and scratchpad memory used, and how are data organized so that they can be read correctly? Are both memories word-addressable?**

**A5)** Conceptually, there are two memories:

- **a main memory**, larger and slower, which stores all input features and kernel filters
- **a scratchpad (SP) memory**, smaller and faster (512 B), used for intermediate data and final outputs.

The movement of data from main memory to the SP is handled by the microcontroller (*i.e., by the C code written in the second part of the lab*). As a consequence, you do not need to worry about how data are arranged in main memory; you only need to define and manage how data are placed in the SP according to your tiling strategy.
**The accelerator accesses only the SP and never directly accesses main memory**.
In general, the workflow is:

1. inputs and filters are stored in main memory;
2. the microcontroller copies a tile of inputs and filters into the SP;
3. the accelerator computes the corresponding partial outputs;
4. the process is repeated for subsequent tiles, accumulating partial results as needed to obtain the final output.